```
import scala.concurrent.{ ExecutionContext, Future }
import akka.http.scaladsl.{ Http, HttpConnectionContext }
import akka.http.scaladsl.model.{ HttpRequest, HttpResponse }

import scala.io.StdIn

object ContainerServer {

  def main(args: Array[String]): Unit = {

    implicit val system =
      ActorSystem(Behaviors.empty, "ContainerServer")

    implicit val ec: ExecutionContext = system.executionContext

    val service: HttpRequest => Future[HttpResponse] =                ❶
      ContainerServiceHandler.withServerReflection(                   ❷
        new ContainerServiceImpl())                                   ❸

    val bindingFuture: Future[Http.ServerBinding] =                   ❹
      Http().newServerAt(
        "localhost",
        8080).bind(service)

    println(s"server at localhost:8080 \nPress RETURN to stop")       ❺
    StdIn.readLine()
    bindingFuture
      .flatMap(_.unbind())
      .onComplete(_ => system.terminate())
  }
}
```

❶ Creates a service to handle request => response

❷ Adds reflection to the handler

❸ ContainerService implementation

❹ Starts the server

❺ Stops the server at Return

Note that there is no reference to serialization here, nor to formatters as in Akka HTTP. This makes things easier.

11.2.5 Running the service